



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
IIC2283 - DISEÑO Y ANÁLISIS DE ALGORITMOS

Ayudantía 2 - Caso Promedio y Cotas Inferiores

Agosto de 2025

Caetano Borges, Manuel Villablanca

1. Algoritmo del mínimo: cotas inferiores y caso promedio

Algorithm 1 Min

```
1: Input: Un arreglo  $A[1 \dots n]$ 
2: Output: El elemento mínimo de  $A[1 \dots n]$ 
3:  $min \leftarrow \infty$ 
4: for  $i = 1$  to  $n$  do
5:   if  $A[i] < min$  then
6:      $min \leftarrow A[i]$ 
7:   end if
8: end for
9: return  $min$ 
```

1. Use árboles de decisión para demostrar una cota inferior para la cantidad de comparaciones necesarias para resolver el problema de encontrar el elemento mínimo de un arreglo.
¿Es esta la mejor cota inferior para el problema?

Solución

Bajo el modelo de árboles de decisión podemos entregar una cota inferior para la cantidad de comparaciones necesarias para resolver el problema si contamos la cantidad de outputs posibles que debería poder entregar el algoritmo. El árbol de decisión tendrá una hoja por cada output posible, y sabemos que su altura cumple con $h \geq \lceil \log_2 \ell \rceil$, donde ℓ es la cantidad de hojas. Con ello, como cada nivel del árbol representa una operación más, la cota inferior para la cantidad de comparaciones necesarias estará dada por el techo del logaritmo de la cantidad de outputs posibles.

Aplicando esto a nuestro problema particular: existen n outputs posibles, ya que el

mínimo podría ser cualquier elemento del arreglo. Luego, en este caso los árboles de decisión nos entregan una cota inferior de $\lceil \log_2 n \rceil$, o en notación asintótica, que todos los algoritmos que resuelven el problema con comparaciones son $\Omega(n)$.

Podemos observar que esta cota es demasiado holgada: sin entrar en demasiado detalle^a es claro que si no revisamos todos los elementos del arreglo puede que no veamos el mínimo, por lo que al menos debemos realizar n operaciones.

^aPara esto se necesitan técnicas de demostración de cotas inferiores como la técnica del adversario, que escapan a nuestro contexto actual

2.Cuál es la complejidad, en torno a las asignaciones a la variable *min*, del:

- a) Mejor caso. ¿Contradice esto a la cota inferior demostrada en el inciso anterior?
- b) Peor caso
- c) Caso promedio

Asuma que todos los elementos son distintos.

Solución

1. Mejor Caso: En el mejor caso, el primer elemento es el mínimo. Cuando ocurre esto, solo realizamos 1 asignación a min. Dado esto, la complejidad es $O(1)$. Notemos que esto no contradice a la cota inferior demostrada en el problema anterior, ya que estamos analizando cosas distintas: antes nos preguntábamos sobre la cantidad de comparaciones necesarias para resolver el problema, y ahora sobre la cantidad de asignaciones a la variable min.
2. Peor Caso: En el peor caso, vamos a tener que visitar cada elemento y cada elemento va a ser más chico que el anterior. Es decir, tenemos una lista ordenada en orden decreciente. Dado este caso, tendríamos que realizar una asignación para cada variable, por lo que tendríamos que hacer n asignaciones. Dado esto, la complejidad es $O(n)$.
3. Caso Promedio: Definimos una variable aleatoria indicatriz X_i , donde la variable nos dice si, en la iteración i , se hace una asignación (1) o no (0). Dado que, si se hace asignación en la iteración i , vamos a hacer 1 asignación. Por lo que la cantidad de asignaciones totales es:

$$\sum_{i=1}^n X_i$$

Por lo que, para encontrar la cantidad esperada de asignaciones, debemos

encontrar el valor esperado de la suma de nuestra indicatriz:

$$\begin{aligned} E[A] &= E \left[\sum_{i=1}^n X_i \right] \\ &= \sum_{i=1}^n E[X_i] \\ &= \sum_{i=1}^n P(X_i = 1) \end{aligned}$$

por linealidad de esperanza

Ahora debemos calcular $P(X_i = 1)$. Es decir, cuál es la probabilidad, estando en la posición i , de que este elemento sea el más chico visto hasta ese momento. En otras palabras, si tomamos un subconjunto $(1, i)$, cuál es la probabilidad que i sea el menor número del subconjunto.

Dada una lista de i elementos, sabemos que hay $i!$ formas de ordenar esos elementos. De esas $i!$ combinaciones fijamos el último elemento como el más chico y podemos generar $(i - 1)!$ permutaciones con los elementos que nos quedan. Dado esto, de las $i!$ permutaciones posibles, $i - 1$ causan que X_i sean 1. Dado esto, tenemos lo siguiente:

$$\begin{aligned} P(X_i = 1) &= \frac{(i - 1)!}{i!} \\ &= \frac{1}{i} \end{aligned}$$

Reemplazando en lo anterior tenemos:

$$\sum_{i=1}^n P(X_i = 1) = \sum_{i=1}^n \frac{1}{i}$$

Donde:

$$\sum_{i=1}^n \frac{1}{i}$$

Es el número armónico H_n el cual se puede aproximar a:

$$\int_1^n \frac{1}{x} dx = \ln(n)$$

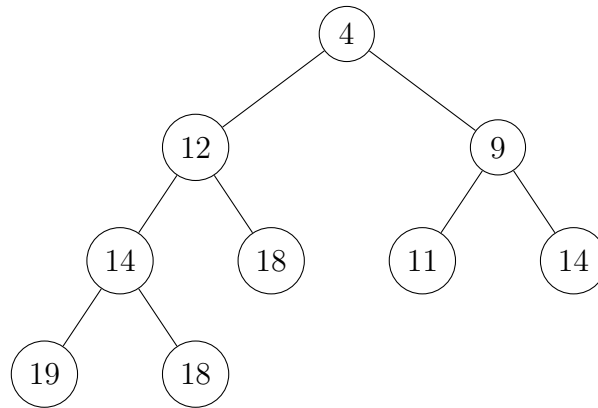
Por lo que:

$$\sum_{i=1}^n P(X_i = 1) \approx \ln(n)$$

Con lo que podemos concluir que la complejidad en torno a asignaciones es $O(\ln(n))$.

2. Caso Promedio Heap

Tal como se estudió en cursos anteriores (como Estructuras de Datos y Algoritmos), un MinHeap es un árbol binario casi lleno (es decir, que todos los niveles, excepto posiblemente el último, están completamente llenos, y los nodos en el último nivel están lo más a la izquierda posible), en el que se cumple que el valor almacenado en cada nodo del árbol es menor o igual que el valor almacenado en sus hijos (si es que estos existen). Así, un Min-Heap de n nodos tiene $\lfloor \lg_2 n \rfloor + 1$ niveles. Un ejemplo de Min-Heap de 10 elementos enteros es el siguiente:



Además de las operaciones típicas de un Min-Heap (insertar, borrar mínimo, obtener mínimo), una operación de interés para muchas aplicaciones es **DecrecerValor** (x, v), para un nodo x y un valor entero v . Si el valor almacenado en el nodo x es $\text{valor}(x)$, la operación modifica ese valor para almacenar $\text{valor}(x) - v$ (es decir, decrementa su valor en v). La operación puede necesitar reestructurar el heap para que se cumpla la condición de orden entre los nodos, posiblemente haciendo “flotar” a x hacia arriba en el árbol, hasta encontrar la posición en donde el orden requerido se cumpla. El algoritmo es el siguiente:

Algorithm 2 DecrecerValor

- 1: **Input:** Un nodo x y un valor v
 - 2: **Efecto:** Almacenar $\text{valor}(x) - v$ en el nodo x .
 - 3: $y \leftarrow x$
 - 4: **while** $y \neq \text{raíz}$ and $\text{valor}(y) < \text{valor}(\text{padre}(y))$ **do**
 - 5: intercambiar valores entre y y $\text{padre}(y)$
 - 6: $y \leftarrow \text{padre}(y)$
 - 7: **end while**
-

Demuestre formalmente que una ejecución de dicho algoritmo realiza $\sim \lfloor \lg_2 n \rfloor$ comparaciones en promedio del tipo “ $\text{valor}(y) < \text{valor}(\text{padre}(y))$ ” (en la línea 4). Para simplificar su análisis, puede asumir que $n = 2^k - 1$, para algún $k \geq 1$. Haga las suposiciones que considere necesarias para que su análisis sea válido.

Solución

Esta pregunta es similar al ejercicio de análisis de caso promedio de la búsqueda binaria. Hay que analizar el algoritmo para todas las posibles invocaciones, esto es, considerando cuánto cuesta invocarlo con cada uno de los elementos del Min-Heap. Asumimos que cada uno de los n elementos tiene probabilidad $1/n$ de ser usado como argumento. (Punto (b) de la rúbrica) Se asume $n = 2^k - 1$, lo cual produce que todos los niveles del Min-Heap estén completamente llenos y simplifica el análisis (pero permitiendo obtener un resultado correcto). Note que en el Min-Heap hay 1 elemento en el nivel 1, 2 elementos en el nivel 2, 4 elementos en el nivel 3, y así siguiente. En general, hay 2^{i-1} elementos en el nivel i , para $1 \leq i \leq \lfloor \lg_2 n \rfloor + 1$.

Supongamos que para una invocación a `DecrecerValor` (x, v), el nodo x está en el nivel i del Min-Heap, para $1 \leq i \leq \lfloor \lg_2 n \rfloor + 1$. Después de decrecer el valor del nodo x , hay que hacerlo flotar tal como indica la iteración principal del algoritmo. Así, x podría terminar en el nivel j , para $1 \leq j \leq i$. La cantidad de comparaciones realizadas va a depender del nivel en que finalice el nodo x luego de ejecutar el algoritmo. Note que si x se mantiene en el mismo nivel i en que estaba, se hace una única comparación en la línea 3. Si x finaliza en el nivel $i - 1$, se hace 2 comparaciones. Y así siguiendo, hasta que si x finaliza en el nivel 2 se hacen $i - 1$ comparaciones. Note que si x termina en el nivel 1 del árbol, también se necesitan $i - 1$ comparaciones. Vamos a asumir que cualquiera de esas cantidades de comparaciones tienen la misma probabilidad de ocurrir, $1/(i - 1)$. Entonces, la cantidad promedio de comparaciones realizadas para un elemento que está en el nivel $i > 1$ está dada por:

$$\frac{1 + 2 + \cdots + i - 1}{i - 1} = \frac{1}{i - 1} \left(\sum_{j=1}^{i-1} j \right) = \frac{i(i - 1)}{2(i - 1)} = \frac{i}{2}$$

Para $i = 1$ el algoritmo hace 0 comparaciones. (Punto (c) de la rúbrica) Ahora consideramos los n posibles nodos del Min-Heap que podrían ser usados como argumento del algoritmo. Dado que asumimos que $n = 2^k - 1$, para $k \geq 1$, y el árbol está lleno en cada nivel, la cantidad promedio de comparaciones realizadas es:

$$\bar{T}(n) = \frac{1}{n} \sum_{i=1}^{\lfloor \lg_2 n \rfloor + 1} 2^{i-1} \cdot \frac{i}{2} = \frac{1}{n} \sum_{i=1}^{\lfloor \lg_2 n \rfloor + 1} 2^{i-2} \cdot i$$

Note que hemos hecho la suma desde $i = 1$, pero hemos dicho anteriormente que $\frac{i}{2}$ no era el valor promedio correcto para el nivel $i = 1$, en donde debería ser 0. Dejaremos la suma así para encontrarle más fácilmente una forma conocida, y al final restaremos $1/2$ al resultado obtenido. (Punto (d) de la rúbrica) Esta última suma tiene la forma de una aritméticogeométrica (ver el Glosario), aunque no es exactamente una de ellas ya que tenemos 2^{i-2} en lugar de 2^i en la suma. Para obtener lo que buscamos, multiplicamos

a ambos lados por 2^2 y tenemos:

$$2^2 \cdot \bar{T}(n) = 2^2 \left(\frac{1}{n} \sum_{i=1}^{\lfloor \lg_2 n \rfloor + 1} 2^{i-2} \cdot i \right) = \frac{1}{n} \sum_{i=1}^{\lfloor \lg_2 n \rfloor + 1} 2^i \cdot i$$

lo cual sí es una aritmético-geométrica que tiene solución:

$$\begin{aligned} 2^2 \cdot \bar{T}(n) &= \frac{1}{n} \sum_{i=1}^{\lfloor \lg_2 n \rfloor + 1} 2^i \cdot i = \frac{1}{n} \cdot \frac{2 - (\lfloor \lg_2 n \rfloor + 2) 2^{\lfloor \lg_2 n \rfloor + 2} + (\lfloor \lg_2 n \rfloor + 1) 2^{\lfloor \lg_2 n \rfloor + 3}}{(1 - 2)^2} \\ &= \frac{2 - 2^2 (n \lfloor \lg_2 n \rfloor + 2n) + 2^3 (n \lfloor \lg_2 n \rfloor + n)}{n} \\ &= \frac{1}{n} \cdot \frac{4n \lfloor \lg_2 n \rfloor + 2}{n} = 4 \lfloor \lg_2 n \rfloor + \frac{2}{n} \end{aligned}$$

Para obtener $\overline{T(n)}$ sólo tenemos que dividir la expresión anterior por 2^2 y se obtiene

$$\bar{T}(n) = \lfloor \lg_2 n \rfloor + \frac{1}{2n}$$

Sólo falta restar $1/2$ (por el caso $i = 1$), por lo que la cantidad promedio de comparaciones del tipo valor $(y) < \text{valor}(\text{padre}(y))$ que realiza el algoritmo es

$$\lfloor \lg_2 n \rfloor + \frac{1}{2n} - \frac{1}{2}$$